Mandatory requirements

1. The implemented patterns must comply with the GoF definition discussed during the courses and laboratories. Incomplete variations or implementations are not accepted.
2. The pattern must be implemented completely correctly to be considered.
3. The solution does not contain compilation errors.
4. Patterns can be treated separately or can be implemented using the same set of classes.
5. Implementations that are not functionally related to the requirements of the subject will NOT be considered (taking any example from other sources will not be evaluated)
6. It is NOT allowed to modify the received classes.
7. Solutions will be cross-checked using MOSS. Code sharing between students is not allowed. Solutions with a degree of similarity of more than 30% will be canceled.

Mandatory Clean Code requirements (you can lose 2 points for each requirement that is not met) - a maximum of 8 points can be lost

1. For naming classes, functions, attributes and variables, the Java Mix CamelCase naming convention shall be used;
2. Patterns and the class containing the main () method are defined in distinct packages that have the form `cts.name.surname.gNrGroup.pattern.X`, `cts.name.surname.gNrGroup.main` (students in the additional year use "AS"Instead of gNrGroup)
3. Classes and methods are implemented respecting clean code principles of KISS, DRY and SOLID (attention to DIP)
4. The class names, methods, variables, and messages displayed on the console must be related to the received subject (generic names are not accepted). Functionally, the methods will display messages to the console that simulate the required action or will implement simple processing.
5. No "magic numbers" and hard-coded values.

3p. Centralized Order Management: You develop software solutions for a smart, tech-enabled coffee shop that prides itself on serving highly customizable drinks to its customers. To ensure consistency across all drink processing stations, there is one and only one **OrderManager** in the entire system. All incoming orders must be routed through this central order processor, which is responsible for Logging orders (based on **IOrderLogging** interface). The service can be used from multiple devices that connect to the same backend server. The **OrderManager** must be designed using **eager instantiation**, because it's initialized when the system starts and stays in memory for the entire operation. Implement a design pattern that does not allow the creation of multiple managers.

0.5p. Implement the ***getNoOfOrders***(), ***logNewOrder***() and ***logCancelOrder***() methods as requested in the **IOrderLogging** comments.

1p. Test the solution by trying to create at least 2 instances for the **OrderManager** and prove that they are the same or show that when you start a new order session, all managers are pointing the same content. Add some orders. Cancel some and print the total number of orders.

3p. Flexible Drink Customization: Every customer can customize their drink by choosing Size (Small, Medium, Large), Type (Espresso, Cappuccino, Latte, Mocha), Milk (Whole, Skimmed, Soy, Almond, Oat), number of sugar spoons, toppings. The solution should allow baristas (and automated systems) to fluently and safely build drink orders with any combination of the drink characteristics. Also, once a drink has been ordered the system does not allow any further modifications. Implement a design pattern that solves the described situation. For Size and Type define enumerations. The drink extends AbstractDrink class.

1.5p. Manually test the implementation by creating at least 3 drinks containing various (different) combinations of characteristics. Prove that a drink once created cannot be modified.